

Session 8 — Gouvernance des projets open source

Objectifs de la séance

- Identifier les modèles de gouvernance des projets majeurs.
- Expliquer le **droit au fork** et ses implications.
- Analyser les mécanismes de prise de décision.
- Comprendre le parcours pour devenir mainteneur.
- Comprendre le rôle des fondations.

Deux actualités en guise d'introduction

Avant d'attaquer la théorie, deux affaires récentes (2026) illustrent les tensions de la gouvernance open source.

Euro Office vs OnlyOffice (mars 2026)

Une coalition de 8 éditeurs européens (Ionos, Nextcloud, XWiki, OpenProject, Abilian...) lance **Euro Office**, fork de la suite **OnlyOffice** sous **AGPL v3**. OnlyOffice crie à la violation de licence et invoque des conditions supplémentaires (Section 7 AGPL) exigeant de conserver logo et marque.

Ce que la Section 7 AGPL autorise réellement :

- **7(b)** — exiger l'**attribution** (créditer l'auteur) : pas forcer l'affichage d'une marque commerciale.
- **7(e)** — **refuser** les droits sur les marques : Euro Office n'utilise pas la marque OnlyOffice, elle a été retirée, ce qui est **conforme**.
- Toute condition supplémentaire non prévue par les clauses 7(a)-(f) est considérée comme une « *further restriction* » que le bénéficiaire de la licence a le **droit de supprimer** (Section 7, dernier paragraphe, confirmé par la Section 10).

Conclusion : (1) droit d'auteur ≠ droit des marques ; (2) on ne peut pas prétendre faire du logiciel libre et refuser un fork. Le fork est **légitime** juridiquement.

TDF expulse les développeurs de Collabora (avril 2026)

The Document Foundation (fondation derrière LibreOffice) expulse **30+ membres** affiliés à **Collabora**, dont **7 des 10 principaux committers** et plusieurs fondateurs du projet.

Les griefs de Collabora (Michael Meeks) : board rempli de non-techniciens, développeurs exclus de la gouvernance, application sélective des politiques de marques, manipulations d'élections. Réponse de Collabora : infrastructure de review indépendante (Gerrit), divergence accrue entre Collabora Office et LibreOffice — signal clair vers un **fork de facto**.

Ces deux cas illustrent les **tensions structurelles** (éditeur commercial vs fondation ; droit au fork vs contrôle de marque) que cette session va éclairer.

Partie 1 — Le droit au fork

Une garantie fondamentale

« *The indispensable ingredient that binds developers together on a free software project is the code's forkability.* » — Karl Fogel, *Producing Open Source Software*.

Définition : droit de copier un projet et de le développer indépendamment. Ce qui le rend possible : licence libre + code source disponible + aucune autorisation nécessaire.

Pourquoi le fork est fondamental

Pour la liberté des utilisateurs : c'est une option de sortie, une garantie d'indépendance vis-à-vis du mainteneur, une protection contre les dérives, et un gage de pérennité du projet.

Pour la gouvernance du projet : le fork joue le rôle d'épée de Damoclès — il force les mainteneurs à écouter la communauté, limite le pouvoir des « dictateurs » et constitue un mécanisme de dernier recours.

Le paradoxe du fork

■ « *The possibility of forks is usually a much greater force than actual forks.* »

Les vrais forks sont rares. Coûts : division de la communauté, duplication des efforts, confusion pour les utilisateurs, perte de momentum. Effet dissuasif : la **menace** suffit souvent à faire avancer les compromis.

Forks célèbres

Fork	Projet original	Raison	Résultat
LibreOffice	OpenOffice	Rachat par Oracle	Succès
MariaDB	MySQL	Rachat par Oracle	Succès
io.js	Node.js	Gouvernance	Réunification
Jenkins	Hudson	Conflit Oracle	Succès
OpenTofu	Terraform	Changement de licence (BSL)	En cours
Valkey	Redis	Changement de licence	En cours
OpenSearch	Elasticsearch	Changement de licence (soutenu par AWS)	Actif

Partie 2 — Modèles de gouvernance

Le modèle BDFL (*Benevolent Dictator For Life*)

Une personne a le dernier mot. Autorité basée sur le mérite, délégation possible. « Dictature » tempérée par le fork.

Exemples : **Linux** (Linus Torvalds), **Python** (Guido van Rossum jusqu'en 2018), **Ubuntu** (Mark Shuttleworth).

■ « *Only when it is clear that no consensus can be reached, and that most of the group wants someone to make a decision so that development can move on, does she put her foot down.* »

Le bon BDFL décide **rarement** par autorité, écoute la communauté, utilise son pouvoir en dernier recours, peut se retirer (comme Guido van Rossum en 2018).

Le modèle Apache (méritocratie)

Structure en cercles concentriques :

Utilisateurs → Contributeurs → Committers (écriture)
→ PMC (Project Management Committee) → Chair

Rôle	Droits	Comment l'obtenir
Utilisateur	Utiliser, reporter des bugs	Automatique
Contributeur	Proposer des patches	Automatique
Committer	Commit direct	Invitation par le PMC
PMC Member	Vote, décisions	Invitation par le PMC
PMC Chair	Représentation officielle	Élection

Principe central : les droits s'obtiennent par le **mérite**, mesuré par la qualité et la régularité des contributions.

Le modèle comité élu

Exemples modernes : **Rust**, **Python** (post-Guido, PEP 13), **Node.js**.

Caractéristiques : *Steering Committee* élu, mandats limités, processus formalisé (RFC), transparence.

Avantages : légitimité démocratique, renouvellement, évite la concentration, processus clairs.

Comparatif

Aspect	BDFL	Méritocratie Apache	Comité élu
Décisions	Rapides	Consensus	Formalisées
Succession	Difficile	Naturelle	Organisée
Scalabilité	Moyenne	Bonne	Bonne
Transparence	Variable	Moyenne	Haute
Exemples	Linux	Apache	Rust, Python

Partie 3 — Prise de décision

Le consensus paresseux (*lazy consensus*)

« *As long as nobody explicitly opposes a proposal, it is recognized as having the support of the community.* »

Fonctionnement : (1) une proposition est faite, (2) période d'attente, (3) si personne ne s'oppose → acceptée, (4) si objection → discussion. Efficace, évite la bureaucratie.

Quand le consensus échoue

Mécanismes de résolution, dans l'ordre :

1. **Discussion prolongée** — chercher un compromis.
2. **Vote** — majorité qualifiée (souvent 2/3 ou 3/4).
3. **Décision BDFL / PMC** — l'autorité tranche.
4. **Report** — attendre plus d'information.
5. **Fork** — dernier recours.

Les RFC / PEP / CEP

Processus formalisé pour les **changements majeurs** :

Projet	Nom	Description
Python	PEP	Python Enhancement Proposal
Rust	RFC	Request for Comments
Kubernetes	KEP	Kubernetes Enhancement Proposal
IETF	RFC	Internet standards

Contenu type : motivation, design, alternatives, impact.

Comparaison avec les ADR (*Architecture Decision Records*) utilisés en entreprise : même logique de documenter les décisions et leurs justifications, mais les PEP/RFC sont **publics** et soumis à la communauté.

Exemple : processus PEP

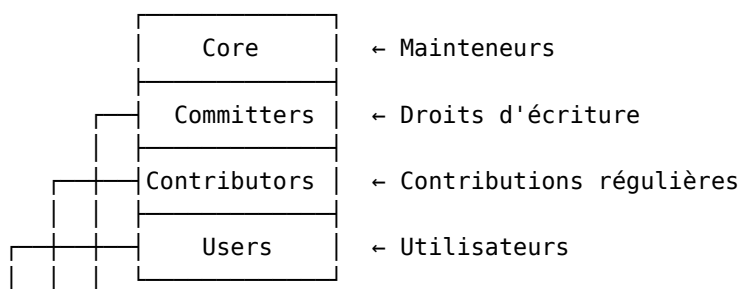
1. Idée → discussion informelle
2. Draft PEP → rédaction formelle
3. Review → commentaires de la communauté

4. Décision → Accepté / Rejeté / Différé
5. Implémentation → si accepté
6. Final → documentation officielle

Types de PEP : *Standards Track* (changements techniques, ex. PEP 484 type hints), *Informational* (recommandations, ex. PEP 8 style guide), *Process* (changements de processus, ex. PEP 13 gouvernance post-Guido).

Partie 4 — Devenir mainteneur

Le modèle *onion*



Progression : de l'extérieur vers l'intérieur, **par le mérite**.

Comment devenir *committer* / mainteneur

Ce qui compte : contributions régulières et de qualité, revue de code des autres, aide aux nouveaux, participation aux discussions, fiabilité et constance dans le temps.

Le processus : cooptation, le plus souvent — proposition par un mainteneur existant, vote ou consensus du groupe, parfois une période d'essai.

Responsabilités

Techniques : revoir les PRs, merger le code, maintenir la qualité, gérer les releases, corriger les bugs critiques.

Communautaires : accueillir les nouveaux, modérer les discussions, communiquer les décisions, représenter le projet à l'extérieur.

Le *burnout* des mainteneurs

Problème croissant. Causes : charge bénévole, pression des utilisateurs (exigences sans retour), critiques non constructives, solitude. Solutions : déléguer, **dire non**, prendre des pauses, chercher des co-mainteneurs. Le *burnout* est au cœur de plusieurs incidents supply chain (cf. XZ Utils, session 10).

Partie 5 — Les fondations

Pourquoi des fondations ?

Besoins des projets : une entité légale, une structure pour détenir marques et noms de domaines, un cadre pour recevoir des dons (avec défiscalisation éventuelle), une protection juridique contre les poursuites.

Ce que les fondations apportent : neutralité (elles ne sont pas une entreprise à objectif commercial), pérennité au-delà des fondateurs, infrastructure partagée (CI, site, événements), gouvernance formelle — parfois au prix d'une certaine lourdeur.

Fondations majeures

Fondation	Style	Projets notables
Apache Software Foundation	Communautaire (<i>Apache Way</i>)	Kafka, Spark, Hadoop, Tomcat
Linux Foundation	<i>Umbrella</i> corporate, budget 500 M\$	Linux, Kubernetes, Node
Mozilla Foundation	<i>Mission-driven</i>	Firefox
Eclipse Foundation	Corporate	Eclipse IDE, Jakarta EE
CNCF (Linux Fdn)	Corporate cloud	Kubernetes, Prometheus, Helm
Python Software Foundation	Communautaire	Python, PyPI
FSF	Idéologique	GNU, GCC

Apache vs Linux Foundation

Apache Software Foundation : devise « *community over code* » ; tous les projets doivent suivre l'*Apache Way* (gouvernance uniforme) ; quasi exclusivement bénévole, staff salarié très réduit (≤ 10 personnes, essentiellement pour l'infra et l'administratif).

Linux Foundation : modèle *umbrella* — les projets hébergés conservent leur propre gouvernance ; financement massif par les grandes entreprises ; budget important (plusieurs centaines de millions de dollars) ; staff conséquent.

Deux philosophies opposées : uniformiser la gouvernance (ASF) vs héberger des projets divers sans imposer un modèle unique (LF).

Associations françaises et européennes

Rappel : **CNLL** (filière professionnelle France), **APELL** (européenne), **APRIL** (grand public), **Framasoft** (logiciels libres + services éthiques grand public), **ADULLACT** (collectivités territoriales).

Ce qu'il faut retenir

1. **Fork** : garantie fondamentale, rarement exercée mais toujours présente.
2. **Modèles** : BDFL, méritocratie Apache, comités élus.
3. **Décisions** : consensus paresseux, votes, RFC/PEP.
4. **Progression** : modèle *onion*, cooptation par le mérite.
5. **Fondations** : structures légales neutres pour les projets — mais toutes ne se valent pas (*Apache Way* vs *umbrella*).
6. **Tensions récurrentes** : éditeur commercial vs fondation (TDF/Collabora), droit au fork vs marque (Euro Office/OnlyOffice) — à suivre en session 9.

Pour aller plus loin

- Karl Fogel, *Producing Open Source Software* : <https://producingoss.com>.
- PEP 13 (gouvernance Python post-Guido) : <https://peps.python.org/pep-0013/>.
- *Apache Way* : <https://www.apache.org/theapacheway/>.